ARQUITECTURA

Sistema de Alarmas y Metricas NOC

NOCBoard Energia v3.9.6

XCIEN Networks - BMAD Method

Version	1.0
Fecha	2026-06-23
Autor	Arquitecto BMAD
Status	Aprobado para implementacion
Proyecto	XCIEN 2.0 Portal - Antigravity
Scope	Alarm log, metricas NOC, analytics dashboard

1. System Overview

1.1 Problem Statement

NOCBoard Energia v3.9.6 monitors 76 power devices (Eltek, Samlex, MEI, ALGCom, Victron) across 20+ cities in Mexico via SNMP polling. It generates real-time alerts (hostOffline, mainsOutage, batteryVoltageLow, etc.) and sends them to Telegram. However:

* No persistent alarm log exists. Alerts are ephemeral -- once a host recovers, the alarm disappears with no his
* No metrics computation. MTTR, MTBF, availability percentages, and trend analysis are impossible without stored
* No alarm correlation. When a VPN trunk fails, 69 hosts cascade offline simultaneously (as evidenced by the 202
* No shift handover context. NOC operators have no structured view of what happened during the previous shift.

1.2 Solution Summary

A three-layer system integrated into the existing XCIEN 2.0 portal:

```
+-----------------------------------------------------------------+
|  LAYER 1: INGESTION                                             |
|  Poller daemon (async thread in servidor_academia.py)           |
|  Polls NOCBoard Energia API (localhost:9404) every 30 seconds   |
|  Detects state transitions -> generates AlarmEvent records      |
+-----------------------------------------------------------------+
         |
         v
+-----------------------------------------------------------------+
|  LAYER 2: PERSISTENCE & COMPUTATION                             |
|  SQLite database (backend/db/alarm_log.db)                      |
|  Tables: alarm_events, alarm_active, host_snapshots, cfe_outages  |
|  Background worker: computes MTTR, MTBF, availability on demand   |
+-----------------------------------------------------------------+
         |
         v
+-----------------------------------------------------------------+
|  LAYER 3: PRESENTATION                                          |
|  New API endpoints under /api/noc/alarms/*                      |
|  Frontend: AlarmMetricsSection.tsx in XCIEN 2.0 portal          |
|  Components: AlarmLog, MetricsDashboard, TrendCharts, TopOffenders|
+-----------------------------------------------------------------+
```

1.3 Design Principles

1. ITU-T X.733 Compliance -- Alarm model follows the X.733 standard for alarm reporting: perceived severity, prob
2. TMN Framework Alignment -- Fault Management (FM) layer of TMN with event-correlation-agent pattern.
3. Brownfield Integration -- All changes fit within the existing monolith. No new processes, no new databases to
4. Scalable to 200+ devices -- SQLite handles millions of rows; index design supports the growth path.
5. Offline-resilient -- If NOCBoard API is unreachable, the poller gracefully degrades using file fallback (exist

2. Data Model

2.1 SQLite Schema

Database file: backend/db/alarm_log.db

```
-- ============================================================
-- ALARM EVENTS (immutable append-only log)
-- Each row = one state transition for one device
-- Follows ITU-T X.733 alarm notification structure
-- ============================================================
CREATE TABLE alarm_events (
    id              INTEGER PRIMARY KEY AUTOINCREMENT,
    event_id        TEXT NOT NULL UNIQUE,           -- UUID v4
    host_id         TEXT NOT NULL,                  -- NOCBoard host ID
    host_ip         TEXT NOT NULL,
    host_name       TEXT NOT NULL,
    city            TEXT NOT NULL,
    site            TEXT NOT NULL DEFAULT '',
    vendor          TEXT NOT NULL DEFAULT '',       -- Eltek, Samlex, MEI, ALGCom, Victron
    device_type     TEXT NOT NULL DEFAULT '',       -- rectifier, inverter, ups, site_monitor
```

```
    -- ITU-T X.733 fields
    alarm_type      TEXT NOT NULL,                  -- see 2.2 Alarm Type Taxonomy
    perceived_severity TEXT NOT NULL DEFAULT 'warning',  -- critical, major, minor, warning, in
    probable_cause  TEXT NOT NULL DEFAULT '',       -- powerProblem, equipmentMalfunction, comm
    specific_problem TEXT NOT NULL DEFAULT '',      -- Human-readable description

    -- State machine
    event_type      TEXT NOT NULL,                  -- raise, clear, change, acknowledge
    correlation_id  TEXT,                           -- Links raise/clear pairs; also groups cas

    -- Timestamps (all UTC ISO-8601)
    event_time      TEXT NOT NULL,                  -- When the event actually occurred
    ingested_at     TEXT NOT NULL DEFAULT (datetime('now')),  -- When we recorded it

    -- NOCBoard source data
    raw_alert_type  TEXT NOT NULL DEFAULT '',       -- Original NOCBoard type: hostOffline, mai
    raw_data        TEXT DEFAULT '{}',              -- Full JSON snapshot from NOCBoard for for

    -- Indexing
    shift_date      TEXT NOT NULL,                  -- YYYY-MM-DD of the shift this event belon
    shift_period    TEXT NOT NULL DEFAULT 'day'     -- day (08:00-20:00) | night (20:00-08:00)
);

CREATE INDEX idx_alarm_events_host ON alarm_events(host_id, event_time);
CREATE INDEX idx_alarm_events_city ON alarm_events(city, event_time);
CREATE INDEX idx_alarm_events_type ON alarm_events(alarm_type, event_time);
CREATE INDEX idx_alarm_events_severity ON alarm_events(perceived_severity, event_time);
CREATE INDEX idx_alarm_events_correlation ON alarm_events(correlation_id);
CREATE INDEX idx_alarm_events_shift ON alarm_events(shift_date, shift_period);
CREATE INDEX idx_alarm_events_raw_type ON alarm_events(raw_alert_type, event_time);

-- ============================================================
-- ACTIVE ALARMS (mutable -- current state of each alarm)
-- One row per currently-active alarm. Deleted when cleared.
-- ============================================================
CREATE TABLE alarm_active (
    id              INTEGER PRIMARY KEY AUTOINCREMENT,
    correlation_id  TEXT NOT NULL UNIQUE,           -- Same as alarm_events.correlation_id for
    host_id         TEXT NOT NULL,
    host_ip         TEXT NOT NULL,
    host_name       TEXT NOT NULL,
    city            TEXT NOT NULL,
    site            TEXT NOT NULL DEFAULT '',
    vendor          TEXT NOT NULL DEFAULT '',
    device_type     TEXT NOT NULL DEFAULT '',

    alarm_type      TEXT NOT NULL,
    perceived_severity TEXT NOT NULL,
    probable_cause  TEXT NOT NULL DEFAULT '',
    specific_problem TEXT NOT NULL DEFAULT '',
    raw_alert_type  TEXT NOT NULL DEFAULT '',

    raised_at       TEXT NOT NULL,                  -- When the alarm was first raised
    last_changed_at TEXT NOT NULL,                  -- Last severity change or re-raise
    acknowledged_at TEXT,                           -- When an operator acknowledged
    acknowledged_by TEXT,                           -- Operator name/ID

    escalation_level INTEGER NOT NULL DEFAULT 0,   -- 0=none, 1=supervisor, 2=manager, 3=direct
    escalated_at    TEXT,
    suppressed      INTEGER NOT NULL DEFAULT 0,    -- 1 if suppressed by correlation engine
    suppression_reason TEXT DEFAULT '',

    notes           TEXT DEFAULT '',                -- Operator notes
    ticket_id       TEXT DEFAULT ''                 -- Link to Odoo project.task or incidente
);

CREATE INDEX idx_alarm_active_host ON alarm_active(host_id);
CREATE INDEX idx_alarm_active_city ON alarm_active(city);
CREATE INDEX idx_alarm_active_severity ON alarm_active(perceived_severity);

-- ============================================================
```

```
-- HOST SNAPSHOTS (periodic state capture for availability calc)
-- One row per host per polling interval
-- ============================================================
CREATE TABLE host_snapshots (
    id              INTEGER PRIMARY KEY AUTOINCREMENT,
    host_id         TEXT NOT NULL,
    host_ip         TEXT NOT NULL,
    city            TEXT NOT NULL,
    status          TEXT NOT NULL,                  -- online, offline, degraded
    health_score    REAL NOT NULL DEFAULT 0,
    latency_ms      REAL DEFAULT NULL,
    packet_loss_pct REAL DEFAULT NULL,
    battery_voltage REAL DEFAULT NULL,
    mains_present   INTEGER DEFAULT NULL,           -- 0/1
    battery_soc     REAL DEFAULT NULL,
    snapshot_time   TEXT NOT NULL DEFAULT (datetime('now')),

    -- Compact: only store if state changed from previous snapshot
    state_changed   INTEGER NOT NULL DEFAULT 0      -- 1 if different from previous snapshot
);

CREATE INDEX idx_snapshots_host_time ON host_snapshots(host_id, snapshot_time);
CREATE INDEX idx_snapshots_city_time ON host_snapshots(city, snapshot_time);
-- Partitioning strategy: DELETE WHERE snapshot_time < date('now', '-90 days') via weekly cron

-- ============================================================
-- CFE OUTAGES (Utility power events -- special tracking)
-- ============================================================
CREATE TABLE cfe_outages (
    id              INTEGER PRIMARY KEY AUTOINCREMENT,
    outage_id       TEXT NOT NULL UNIQUE,
    city            TEXT NOT NULL,
    site            TEXT NOT NULL DEFAULT '',
    affected_hosts  TEXT NOT NULL DEFAULT '[]',     -- JSON array of host_ids
    started_at      TEXT NOT NULL,
    ended_at        TEXT,                           -- NULL = ongoing
    duration_minutes REAL DEFAULT NULL,             -- Computed on close
    detected_by     TEXT NOT NULL DEFAULT 'snmp',   -- snmp, manual, correlation
    correlation_id  TEXT                            -- Links to alarm_events
);

CREATE INDEX idx_cfe_city ON cfe_outages(city, started_at);

-- ============================================================
-- METRICS CACHE (pre-computed metrics for fast dashboard rendering)
-- ============================================================
CREATE TABLE metrics_cache (
    id              INTEGER PRIMARY KEY AUTOINCREMENT,
    metric_key      TEXT NOT NULL,                  -- e.g., "mttr:ciudad:monterrey:2026-06-23"
    metric_type     TEXT NOT NULL,                  -- mttr, mtbf, availability, alarm_freq
    scope           TEXT NOT NULL DEFAULT 'global', -- global, city, host, vendor
    scope_value     TEXT NOT NULL DEFAULT '',        -- "monterrey", "10.1.1.1", "eltek"
    period          TEXT NOT NULL,                  -- daily, weekly, monthly
    period_start    TEXT NOT NULL,
    period_end      TEXT NOT NULL,
    value           REAL NOT NULL,
    metadata        TEXT DEFAULT '{}',              -- Additional JSON context
    computed_at     TEXT NOT NULL DEFAULT (datetime('now')),

    UNIQUE(metric_key)
);

CREATE INDEX idx_metrics_type_scope ON metrics_cache(metric_type, scope, scope_value, period_st
```

2.2 Alarm Type Taxonomy (ITU-T X.733 + Telecom NOC)

```
ALARM_TYPE_MAP = {
    # NOCBoard raw type -> (alarm_type, probable_cause, default_severity)

    # Communications alarms
    "hostOffline":          ("communicationsAlarm",  "communicationsSubsystemFailure", "critica
    "hostRecovered":        ("communicationsAlarm",  "communicationsSubsystemFailure", "cleared
```

```
    "highLatency":          ("qualityOfServiceAlarm","responseTimeExcessive",          "warning

    # Environmental alarms
    "mainsOutage":          ("environmentalAlarm",   "powerProblem",                   "major")
    "siteOnBatteryBackup":  ("environmentalAlarm",   "powerProblem",                   "major")

    # Equipment alarms
    "batteryVoltageLow":    ("equipmentAlarm",       "batteryChargingFailure",         "major")
    "batterySOCLow":        ("equipmentAlarm",       "batteryChargingFailure",         "critica
    "siteCritical":         ("equipmentAlarm",       "equipmentMalfunction",           "critica

    # Processing errors (future)
    "snmpPollFailed":       ("processingErrorAlarm", "softwareProgramError",           "minor")
    "configMismatch":       ("processingErrorAlarm", "configurationMismatch",          "warning
}

SEVERITY_LEVELS = {
    "critical":      5,   # Service-affecting, immediate action required
    "major":         4,   # Service-degrading, action required within 15 min
    "minor":         3,   # Non-service-affecting, action within 4 hours
    "warning":       2,   # Potential problem, monitor
    "indeterminate": 1,   # Cannot determine severity
    "cleared":       0,   # Alarm condition no longer present
}
```

2.3 State Machine

```
                    +--------+
                    |  IDLE  | (no alarm)
                    +---+----+
                        |
              [condition detected]
                        |
                        v
                   +---------+
          +------->| RAISED  |<-----------+
          |        +----+----+            |
          |             |                 |
  [re-raised]   [operator ack]    [severity change]
          |             |                 |
          |             v                 |
          |      +-----------+            |
          +------| ACKNOWLEDGED|----------+
                 +-----+-----+
                       |
              [condition cleared]
                        |
                        v
                  +---------+
                  | CLEARED |
                  +---------+
                        |
                  [after TTL]
                        |
                        v
                  (removed from
                   alarm_active)
```

3. Ingestion Engine

3.1 Poller Architecture

The poller runs as a daemon thread inside servidor_academia.py, following the same pattern as the existing _nocboard_watchdog() thread (line 1301).

```
# ---- File: backend/alarm_ingestion.py ----

"""
Alarm Ingestion Engine for NOCBoard Energia.

Polls NOCBoard Energia API (localhost:9404) every 30 seconds.
```

```
Detects state transitions by comparing current poll with previous state.
Generates alarm_event records for every transition.
Manages alarm_active table for current alarm state.

Threading model: single daemon thread with its own SQLite connection
(SQLite WAL mode allows concurrent reads from FastAPI request threads).
"""

import sqlite3
import threading
import time
import uuid
import json
import logging
import requests
from datetime import datetime, timezone, timedelta
from typing import Dict, List, Optional, Tuple
from collections import defaultdict

logger = logging.getLogger("XCIEN-ALARM-INGESTION")

# Configuration
POLL_INTERVAL_SECONDS = 30
ENERGIA_API_URL = "http://localhost:9404/api"
ENERGIA_API_KEY = "f4f5ef40c4c54aeca1d6a66109e4555d"
ENERGIA_HOSTS_FALLBACK = "~/Library/Application Support/NOCBoardEnergia/hosts.json"
DB_PATH = "db/alarm_log.db"

# Correlation windows
CASCADE_WINDOW_SECONDS = 120   # Alarms within 2 min of first = potential cascade
CASCADE_THRESHOLD = 5          # 5+ hosts in window = cascade event
CLEAR_HOLD_MINUTES = 5         # Keep cleared alarms in active table for 5 min
```

3.2 State Transition Detection

The poller maintains an in-memory map of {host_id: previous_state}. On each poll cycle:

```
For each host in current poll:
    previous = state_map.get(host_id)
    current  = derive_state(host)

    if previous is None:
        # First time seeing this host -- set baseline, no event
        state_map[host_id] = current
        continue

    if current != previous:
        # STATE TRANSITION DETECTED
        if current.status == "offline" and previous.status == "online":
            emit_event(type="raise", alarm_type=classify(host))
        elif current.status == "online" and previous.status == "offline":
            emit_event(type="clear", correlation_id=find_open_alarm(host_id))
        elif current.severity != previous.severity:
            emit_event(type="change", ...)

        state_map[host_id] = current
```

State derivation from NOCBoard data:

```
def derive_host_state(host: dict) -> HostState:
    """Extract alarm-relevant state from a NOCBoard host object."""
    ping = host.get("ping", host.get("lastPingResult", {}))
    metrics = host.get("_api_metrics", host.get("latestMetrics", {}))

    status = host.get("status", "unknown")
    alerts = []

    # Check mains/CFE
    mains = metrics.get("mains_present", metrics.get("mainsPresent"))
    if mains is not None and not mains:
        alerts.append("mainsOutage")

    # Check battery
    batt_v = metrics.get("battery_voltage", metrics.get("batteryVoltage"))
```

```
    if batt_v is not None and batt_v < 44.0:  # 48V system threshold
        alerts.append("batteryVoltageLow")

    batt_soc = metrics.get("battery_soc", metrics.get("batterySOC"))
    if batt_soc is not None and batt_soc < 20.0:
        alerts.append("batterySOCLow")

    # Check latency
    latency = ping.get("latency_avg", ping.get("latencyAvg", 0))
    if latency > 200:
        alerts.append("highLatency")

    return HostState(
        status=status,
        alerts=frozenset(alerts),
        health_score=host.get("health_score", host.get("healthScore", 0)),
        mains_present=mains,
        battery_voltage=batt_v,
        battery_soc=batt_soc,
        latency_ms=latency,
    )
```

3.3 NOCBoard Events API Integration

NOCBoard Energia exposes /api/events which contains its own event log. The poller also ingests these as a secondary source:

```
def poll_nocboard_events() -> List[dict]:
    """Fetch events from NOCBoard's native event stream."""
    try:
        r = requests.get(
            f"{ENERGIA_API_URL}/events",
            headers={"X-API-Key": ENERGIA_API_KEY},
            timeout=5
        )
        if r.ok:
            data = r.json()
            return data.get("events", data) if isinstance(data, dict) else data
    except Exception as e:
        logger.warning(f"Could not fetch NOCBoard events: {e}")
    return []
```

The poller reconciles both sources: its own state-transition detection (authoritative for timing) and NOCBoard's native events (authoritative for alarm type classification like mainsOutage vs hostOffline).

3.4 Alarm Correlation Engine

Purpose: Suppress cascading alerts when a trunk/VPN failure causes mass device offline events (like the 2026-06-18 incident: 69 hosts, 19 cities, 2,954 events in 8 minutes).

```
class CorrelationEngine:
    """
    Rule-based alarm correlation following TMN guidelines.

    Rules:
    1. CASCADE: If N+ hosts go offline within WINDOW seconds,
       group under a single correlation_id, suppress individual Telegram alerts,
       send one summary alert instead.

    2. CHILD_SUPPRESSION: If a site's upstream link is down,
       suppress alarms from devices behind that link.

    3. FLAP_DAMPING: If a host oscillates online/offline more than
       3 times in 10 minutes, suppress until stable for 5 minutes.

    4. CFE_GROUPING: If mainsOutage fires for multiple hosts in the
       same city within 5 minutes, group as a single CFE outage event.
    """

    def __init__(self):
        self.pending_raises: List[AlarmEvent] = []
        self.flap_counters: Dict[str, List[float]] = defaultdict(list)
        self.cascade_window_start: Optional[float] = None
```

```
    def process(self, event: AlarmEvent) -> CorrelationResult:
        now = time.time()

        # --- Flap damping ---
        if event.event_type == "raise":
            flaps = self.flap_counters[event.host_id]
            flaps.append(now)
            # Keep only last 10 minutes
            flaps[:] = [t for t in flaps if now - t < 600]
            if len(flaps) >= 6:  # 3 raise+clear cycles
                return CorrelationResult(
                    action="suppress",
                    reason=f"flap_damping: {len(flaps)} transitions in 10min",
                    correlation_id=f"FLAP-{event.host_id}-{int(now)}"
                )

        # --- Cascade detection ---
        if event.event_type == "raise" and event.raw_alert_type in ("hostOffline", "siteCritica
            self.pending_raises.append(event)
            # Flush events older than the cascade window
            cutoff = now - CASCADE_WINDOW_SECONDS
            self.pending_raises = [e for e in self.pending_raises if e.timestamp > cutoff]

            if len(self.pending_raises) >= CASCADE_THRESHOLD:
                cities = set(e.city for e in self.pending_raises)
                cascade_id = f"CASCADE-{int(now)}"
                return CorrelationResult(
                    action="cascade",
                    reason=f"cascade: {len(self.pending_raises)} hosts across {len(cities)} cit
                    correlation_id=cascade_id,
                    affected_events=list(self.pending_raises)
                )

        # --- CFE grouping ---
        if event.raw_alert_type == "mainsOutage":
            # Look for other mainsOutage events in same city within 5 min
            same_city_cfe = [
                e for e in self.pending_raises
                if e.city == event.city
                and e.raw_alert_type == "mainsOutage"
                and abs(e.timestamp - event.timestamp) < 300
            ]
            if same_city_cfe:
                cfe_id = same_city_cfe[0].correlation_id or f"CFE-{event.city}-{int(now)}"
                return CorrelationResult(
                    action="group_cfe",
                    reason=f"cfe_grouping: {len(same_city_cfe)+1} devices in {event.city}",
                    correlation_id=cfe_id
                )

        # --- No correlation needed ---
        return CorrelationResult(action="pass", correlation_id=str(uuid.uuid4()))
```

3.5 Snapshot Strategy

To avoid storing 76 snapshots every 30 seconds (219,648 rows/day), use change-detection compression:

```
Full snapshot:  every 5 minutes (76 hosts x 288 intervals/day = 21,888 rows/day)
Delta snapshot: only when state_changed=1 (estimated 50-200 rows/day)
Retention:      90 days of full snapshots, 365 days of delta snapshots
Purge:          Weekly background job deletes old full snapshots
```

Estimated storage: ~50MB/year for 76 hosts. Well within SQLite's capabilities.

4. API Design

4.1 Endpoint Catalog

All endpoints are under /api/noc/alarms/ and must be registered in servidor_academia.py BEFORE the SPA catch-all at line 6495.

```
# ============================================================
```

```
# ALARM LOG (read alarm history)
# ================================================================

GET  /api/noc/alarms/events
     Query params:
       - start_date: ISO-8601 (default: 24h ago)
       - end_date:   ISO-8601 (default: now)
       - city:       filter by city name
       - host_id:    filter by host
       - alarm_type: filter by type
       - severity:   filter by severity
       - limit:      max results (default: 500, max: 5000)
       - offset:     pagination offset
     Returns: { total, events: AlarmEvent[], filters_applied }

GET  /api/noc/alarms/events/{event_id}
     Returns: full AlarmEvent with raw_data

GET  /api/noc/alarms/active
     Returns: { total, alarms: AlarmActive[] }
     (Currently active/unresolved alarms)

POST /api/noc/alarms/active/{correlation_id}/acknowledge
     Body: { operator: string, notes?: string }
     Returns: updated AlarmActive

# ================================================================
# METRICS (computed KPIs)
# ================================================================

GET  /api/noc/alarms/metrics/summary
     Query params:
       - period: "24h" | "7d" | "30d" | "90d"
       - city?: string
       - vendor?: string
     Returns: {
       mttr_minutes:     float,
       mtbf_hours:       float,
       availability_pct: float,
       total_alarms:     int,
       critical_alarms:  int,
       cfe_outages:      int,
       avg_cfe_duration_min: float,
       alarm_rate_per_day: float,
       top_alarm_types:  [{type, count, pct}],
     }

GET  /api/noc/alarms/metrics/availability
     Query params:
       - period: "24h" | "7d" | "30d"
       - group_by: "city" | "host" | "vendor"
     Returns: [{
       name:           string,
       availability_pct: float,
       total_downtime_min: float,
       incidents:      int,
       sla_status:     "met" | "at_risk" | "breached"
     }]

GET  /api/noc/alarms/metrics/mttr
     Query params:
       - period, city, vendor, alarm_type
     Returns: {
       overall_mttr_min: float,
       by_severity: { critical, major, minor, warning },
       by_city:     [{city, mttr_min}],
       by_vendor:   [{vendor, mttr_min}],
       trend:       [{date, mttr_min}],  -- daily trend
     }

GET  /api/noc/alarms/metrics/mtbf
     Query params: period, city, vendor
```

```
     Returns: {
       overall_mtbf_hours: float,
       by_city:   [{city, mtbf_hours}],
       by_vendor: [{vendor, mtbf_hours}],
       trend:     [{date, mtbf_hours}],
     }

GET  /api/noc/alarms/metrics/trends
     Query params:
       - period: "7d" | "30d" | "90d"
       - granularity: "hourly" | "daily" | "weekly"
       - city?, vendor?, alarm_type?
     Returns: {
       data_points: [{
         timestamp: string,
         alarm_count: int,
         critical_count: int,
         cfe_count: int,
         avg_resolution_min: float,
       }]
     }

# ================================================================
# TOP OFFENDERS
# ================================================================

GET  /api/noc/alarms/top-offenders
     Query params:
       - period: "7d" | "30d" | "90d"
       - limit: int (default: 10)
       - group_by: "host" | "city" | "site"
     Returns: [{
       name:         string,
       alarm_count:  int,
       critical_pct: float,
       avg_mttr_min: float,
       downtime_min: float,
       trend:        "improving" | "stable" | "degrading"
     }]

# ================================================================
# CFE OUTAGES
# ================================================================

GET  /api/noc/alarms/cfe-outages
     Query params: period, city
     Returns: {
       total: int,
       outages: [{
         outage_id, city, site, started_at, ended_at,
         duration_minutes, affected_host_count,
       }],
       stats: {
         avg_duration_min, max_duration_min,
         most_affected_city, outages_per_week,
       }
     }

# ================================================================
# SHIFT HANDOVER
# ================================================================

GET  /api/noc/alarms/shift-report
     Query params:
       - shift_date: YYYY-MM-DD (default: today)
       - shift_period: "day" | "night" (default: current)
     Returns: {
       shift: { date, period, start, end },
       summary: {
         alarms_raised, alarms_cleared, alarms_still_active,
         critical_events, cfe_outages,
         mttr_this_shift_min,
```

```
      },
      active_alarms:    AlarmActive[],
      notable_events:   AlarmEvent[],   -- critical + major only
      cfe_events:       CfeOutage[],
      host_changes:     [{host, from_status, to_status, time}],
      recommendations:  string[],       -- auto-generated based on patterns
    }

# ================================================================
# ESCALATION
# ================================================================

POST /api/noc/alarms/active/{correlation_id}/escalate
     Body: { level: 1|2|3, reason: string }
     Returns: updated AlarmActive

POST /api/noc/alarms/active/{correlation_id}/notes
     Body: { text: string, operator: string }
     Returns: updated AlarmActive

# ================================================================
# SLA TRACKING
# ================================================================

GET  /api/noc/alarms/sla
     Query params: period, city
     Returns: {
       targets: { availability_pct: 99.5, mttr_critical_min: 30, mttr_major_min: 120 },
       actual: {
         availability_pct, mttr_critical_min, mttr_major_min,
         sla_met: boolean, breaches: [{ type, target, actual, when }]
       },
       by_city: [{ city, availability_pct, sla_status }]
     }
```

4.2 Pydantic Models

```
# ---- File: backend/alarm_models.py ----

from pydantic import BaseModel
from typing import List, Optional, Dict, Any
from datetime import datetime

class AlarmEventResponse(BaseModel):
    event_id: str
    host_id: str
    host_ip: str
    host_name: str
    city: str
    site: str
    vendor: str
    device_type: str
    alarm_type: str
    perceived_severity: str
    probable_cause: str
    specific_problem: str
    event_type: str            # raise, clear, change, acknowledge
    correlation_id: Optional[str]
    event_time: str
    raw_alert_type: str
    shift_date: str
    shift_period: str

class AlarmActiveResponse(BaseModel):
    correlation_id: str
    host_id: str
    host_ip: str
    host_name: str
    city: str
    site: str
    vendor: str
    device_type: str
```

```
    alarm_type: str
    perceived_severity: str
    probable_cause: str
    specific_problem: str
    raw_alert_type: str
    raised_at: str
    duration_minutes: float     # Computed: now - raised_at
    last_changed_at: str
    acknowledged_at: Optional[str]
    acknowledged_by: Optional[str]
    escalation_level: int
    suppressed: bool
    suppression_reason: str
    notes: str
    ticket_id: str

class MetricsSummaryResponse(BaseModel):
    period: str
    mttr_minutes: float
    mtbf_hours: float
    availability_pct: float
    total_alarms: int
    critical_alarms: int
    cfe_outages: int
    avg_cfe_duration_min: float
    alarm_rate_per_day: float
    top_alarm_types: List[Dict[str, Any]]
    computed_at: str

class AcknowledgeRequest(BaseModel):
    operator: str
    notes: Optional[str] = ""

class EscalateRequest(BaseModel):
    level: int           # 1=supervisor, 2=manager, 3=director
    reason: str

class NoteRequest(BaseModel):
    text: str
    operator: str
```

5. Metric Calculations

5.1 MTTR (Mean Time To Repair/Recover)

```
MTTR = SUM(resolution_time - raise_time) / COUNT(resolved_alarms)

Where:
  resolution_time = alarm_events.event_time WHERE event_type='clear' AND correlation_id=X
  raise_time      = alarm_events.event_time WHERE event_type='raise' AND correlation_id=X

SQL:
  SELECT AVG(
    (julianday(clear_ev.event_time) - julianday(raise_ev.event_time)) * 24 * 60
  ) as mttr_minutes
  FROM alarm_events raise_ev
  JOIN alarm_events clear_ev
    ON raise_ev.correlation_id = clear_ev.correlation_id
  WHERE raise_ev.event_type = 'raise'
    AND clear_ev.event_type = 'clear'
    AND raise_ev.event_time >= :start_date
    AND raise_ev.event_time <= :end_date

Segmented by severity:
  - Critical: Target < 30 min
  - Major:    Target < 2 hours
  - Minor:    Target < 8 hours
  - Warning:  Target < 24 hours
```

5.2 MTBF (Mean Time Between Failures)

```
MTBF = Total_operational_time / Number_of_failures

Per host:
  operational_periods = time intervals where host was online
  failures = COUNT of 'raise' events for that host
  MTBF = SUM(operational_periods) / failures

SQL:
  WITH host_failures AS (
    SELECT host_id, COUNT(*) as failure_count,
           MIN(event_time) as first_failure,
           MAX(event_time) as last_failure
    FROM alarm_events
    WHERE event_type = 'raise'
      AND alarm_type = 'communicationsAlarm'
      AND event_time >= :start_date
    GROUP BY host_id
    HAVING failure_count > 1
  )
  SELECT host_id,
    (julianday(last_failure) - julianday(first_failure)) * 24 / (failure_count - 1) as mtbf_hou
  FROM host_failures

Global MTBF:
  total_host_hours = host_count * period_hours
  total_failures = COUNT(raise events in period)
  MTBF = total_host_hours / total_failures
```

5.3 Availability

```
Availability % = (Total_time - Downtime) / Total_time * 100

Per host:
  Total_time = period end - period start (in minutes)
  Downtime   = SUM of all offline durations in that period

  Offline duration = clear_time - raise_time for each alarm

  For currently-active alarms: clear_time = NOW()

SQL (using snapshots for precision):
  SELECT host_id,
    COUNT(CASE WHEN status = 'online' THEN 1 END) * 100.0 / COUNT(*) as availability_pct
  FROM host_snapshots
  WHERE snapshot_time >= :start_date
    AND snapshot_time <= :end_date
  GROUP BY host_id

Per city:
  Average availability of all hosts in that city.

Per network (overall):
  Weighted average by host criticality (future: assign weights per host).
  Default: simple average across all 76 hosts.

SLA Thresholds:
  - Gold:   99.9% (43.8 min downtime/month)
  - Silver: 99.5% (3.65 hours downtime/month)
  - Bronze: 99.0% (7.3 hours downtime/month)
  - XCIEN Target: 99.5% (aligns with Telcel/CFE standards in Mexico)
```

5.4 Alarm Frequency

```
Alarm Rate = COUNT(raise events) / Period_in_days

By type:  GROUP BY alarm_type
By city:  GROUP BY city
By vendor: GROUP BY vendor
By hour:  GROUP BY strftime('%H', event_time)  -- identifies peak hours
```

5.5 CFE Outage Analysis

```
CFE metrics:
  - Frequency:     COUNT(cfe_outages) per period
  - Avg duration:  AVG(duration_minutes)
  - Max duration:  MAX(duration_minutes)
  - Impact:        AVG(affected_host_count)
  - Worst city:    city with MAX(cfe_outage_count)
  - Pattern:       GROUP BY strftime('%H', started_at) -- identifies CFE peak hours
```

6. Frontend Component Architecture

6.1 File Structure

```
src/pages/xcien2/sections/
  AlarmMetricsSection.tsx          -- Main section container (registered in index.tsx)
  alarm-metrics/
    AlarmLog.tsx                   -- Filterable event log table
    ActiveAlarms.tsx               -- Current active alarms with actions
    MetricsDashboard.tsx           -- KPI cards + summary
    AvailabilityChart.tsx          -- Availability % by city/host (bar chart)
    TrendChart.tsx                 -- Time-series alarm trends (line chart)
    TopOffenders.tsx               -- Ranked list of worst-performing sites
    CfeOutageTracker.tsx           -- CFE outage timeline + stats
    ShiftReport.tsx                -- Shift handover summary
    SlaTracker.tsx                 -- SLA compliance dashboard
    hooks/
      useAlarmData.ts              -- Data fetching + polling hook
      useMetrics.ts                -- Metrics API hook with caching
    types.ts                       -- TypeScript interfaces
    constants.ts                   -- Colors, thresholds, severity maps
    utils.ts                       -- Formatters, duration calculators
```

6.2 Component Hierarchy

```
AlarmMetricsSection
  |-- TabBar ("Dashboard" | "Alarm Log" | "Active Alarms" | "CFE" | "SLA" | "Shift Report")
  |
  |-- [Tab: Dashboard]
  |     |-- KpiRow
  |     |     |-- KpiCard (MTTR)
  |     |     |-- KpiCard (MTBF)
  |     |     |-- KpiCard (Availability %)
  |     |     |-- KpiCard (Active Alarms)
  |     |     |-- KpiCard (CFE Outages 30d)
  |     |     +-- KpiCard (Alarm Rate /day)
  |     |
  |     |-- AvailabilityChart (horizontal bars by city, color-coded by SLA)
  |     |
  |     |-- Row
  |     |     |-- TrendChart (alarm count trend, 30d, daily granularity)
  |     |     +-- AlarmTypeDonut (pie chart by alarm type)
  |     |
  |     |-- TopOffenders (ranked table, top 10 sites)
  |     +-- VendorBreakdown (bar chart: alarms by vendor)
  |
  |-- [Tab: Alarm Log]
  |     |-- FilterBar (date range, city, severity, type, host search)
  |     |-- AlarmLog (paginated table with expandable rows)
  |     +-- ExportButton (CSV download)
  |
  |-- [Tab: Active Alarms]
  |     |-- SeverityFilter (critical | major | minor | all)
  |     |-- ActiveAlarms (cards with acknowledge/escalate/notes actions)
  |     +-- CorrelationGroups (collapsed cascading alarms)
  |
  |-- [Tab: CFE]
  |     |-- CfeStats (KPI cards: count, avg duration, worst city)
  |     |-- CfeTimeline (gantt-style timeline of outages)
```

```
  |     +-- CfeHeatmap (outages by hour-of-day / day-of-week)
  |
  |-- [Tab: SLA]
  |     |-- SlaTargets (configured targets display)
  |     |-- SlaCityTable (availability by city with traffic-light status)
  |     +-- SlaBreaches (list of SLA violations)
  |
  +-- [Tab: Shift Report]
        |-- ShiftSelector (date + day/night)
        |-- ShiftSummary (KPIs for the shift)
        |-- ShiftTimeline (chronological notable events)
        |-- ActiveAtHandover (alarms still active when shift ended)
        +-- ShiftExport (generate PDF for Telegram)
```

6.3 Visualization Strategy

All charts rendered with inline SVG (no external chart library dependency), following the existing pattern in VentasSection.tsx (sparkbars, donuts). This keeps the bundle small and avoids adding recharts/chart.js.

```
// Color palette (extends existing NocSection.tsx palette)
const ALARM_COLORS = {
  critical:  '#ff3366',   // Red -- matches NOC palette
  major:     '#ff8800',   // Orange
  minor:     '#ffcc00',   // Yellow
  warning:   '#88aaff',   // Light blue
  cleared:   '#00ff88',   // Green -- matches NOC palette

  // SLA status
  sla_met:      '#00ff88',
  sla_at_risk:  '#ffcc00',
  sla_breached: '#ff3366',

  // CFE
  cfe_outage: '#ff5500',
  cfe_normal: '#00cc66',
};
```

6.4 Data Fetching Pattern

```
// hooks/useAlarmData.ts

function useAlarmData(period: string = '24h') {
  const [summary, setSummary] = useState<MetricsSummary | null>(null);
  const [activeAlarms, setActiveAlarms] = useState<AlarmActive[]>([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const fetchData = async () => {
      const [summaryRes, activeRes] = await Promise.all([
        fetch(`/api/noc/alarms/metrics/summary?period=${period}`),
        fetch('/api/noc/alarms/active'),
      ]);
      setSummary(await summaryRes.json());
      setActiveAlarms((await activeRes.json()).alarms);
      setLoading(false);
    };

    fetchData();
    const interval = setInterval(fetchData, 30_000); // Match NOCBoard poll rate
    return () => clearInterval(interval);
  }, [period]);

  return { summary, activeAlarms, loading };
}
```

6.5 Section Registration

In src/pages/xcien2/index.tsx, add the new section to the sidebar configuration:

```
// In the SECTIONS array, add under the "Infraestructura" group:
{
  id: 'alarm-metrics',
  label: 'Alarmas y Metricas',
```

```
  icon: '// bell icon SVG',
  group: 'infraestructura',
  component: lazy(() => import('./sections/AlarmMetricsSection')),
}
```

7. Severity Escalation Rules

7.1 Time-Based Escalation Matrix

```
+------------------+-------------------+-------------------+-------------------+
| Severity         | Level 1           | Level 2           | Level 3           |
|                  | (NOC Operator)    | (NOC Supervisor)  | (NOC Manager/Dir) |
+------------------+-------------------+-------------------+-------------------+
| Critical (P1)    | Immediate         | +15 min           | +30 min           |
| Major (P2)       | Immediate         | +30 min           | +2 hours          |
| Minor (P3)       | +15 min           | +2 hours          | +8 hours          |
| Warning (P4)     | +1 hour           | +4 hours          | +24 hours         |
+------------------+-------------------+-------------------+-------------------+

Actions per level:
  Level 0: Alarm raised, appears in Active Alarms dashboard
  Level 1: Telegram notification to NOCBOARD ENERGIA channel
  Level 2: Telegram notification to NOC supervisor + direct message
  Level 3: Telegram notification to NOC manager + email
```

7.2 Automatic Escalation Logic

```
def check_escalation(alarm: AlarmActive, now: datetime) -> Optional[int]:
    """Returns new escalation level if escalation is due, else None."""
    raised = datetime.fromisoformat(alarm.raised_at)
    elapsed_min = (now - raised).total_seconds() / 60

    if alarm.acknowledged_at:
        # If acknowledged, don't auto-escalate
        return None

    sev = alarm.perceived_severity
    current_level = alarm.escalation_level

    thresholds = {
        "critical": [0, 15, 30],      # L1 at 0min, L2 at 15min, L3 at 30min
        "major":    [0, 30, 120],
        "minor":    [15, 120, 480],
        "warning":  [60, 240, 1440],
    }

    levels = thresholds.get(sev, thresholds["warning"])
    for level, threshold_min in enumerate(levels):
        if elapsed_min >= threshold_min and current_level < level + 1:
            return level + 1

    return None
```

7.3 Notification Routing

```
Escalation Level 1:
  - Telegram: @xcien_nocboard_bot -> NOCBOARD ENERGIA channel (-1003763039964)
  - Dashboard: alarm appears in Active Alarms tab

Escalation Level 2:
  - Telegram: @jmmc2026_bot -> NOC supervisor direct message
  - Dashboard: alarm gets orange highlight
  - Auto-create Odoo project.task (if not already linked)

Escalation Level 3:
  - Telegram: @jmmc2026_bot -> Director/Manager direct message
  - Dashboard: alarm gets red pulsing highlight
  - Odoo task: priority escalated to "urgent"
```

8. Shift Handover Protocol

8.1 Shift Definitions

```
SHIFTS = {
    "day":   {"start": "08:00", "end": "20:00", "label": "Turno Dia"},
    "night": {"start": "20:00", "end": "08:00", "label": "Turno Noche"},
}
# Timezone: America/Monterrey (CST/CDT)
```

8.2 Auto-Generated Report Content

The /api/noc/alarms/shift-report endpoint generates:

1. Summary KPIs: alarms raised/cleared/active, MTTR for the shift, CFE events
2. Notable Events: all critical and major alarms with timeline
3. Active at Handover: alarms still unresolved when the shift ends
4. CFE Outages: any utility power events during the shift
5. Status Changes: hosts that changed state (online<->offline)
6. Recommendations: auto-generated from patterns:
    * "Site X has had 3 alarms in 4 hours -- investigate root cause"
    * "Battery SOC below 30% at sites Y, Z -- schedule maintenance"
    * "CFE outage in Monterrey lasted 45 min -- verify generator readiness"

8.3 PDF Export

Uses existing xcien_pdf_template.py (XcienPDF class) for institutional-format PDF generation:

```
@app.get("/api/noc/alarms/shift-report/pdf")
def generate_shift_report_pdf(shift_date: str, shift_period: str):
    """Generate PDF shift report and optionally send via Telegram."""
    report_data = _build_shift_report(shift_date, shift_period)

    pdf = XcienPDF(
        title=f"Reporte de Turno - {report_data['shift']['label']}",
        subtitle=f"{shift_date}",
    )
    # ... build PDF sections
    pdf_path = pdf.save()

    # Send to Telegram via @jmmc2026_bot
    bot = TelegramBot(...)
    bot.send_document(pdf_path, caption=f"Shift Report {shift_date} {shift_period}")

    return FileResponse(pdf_path)
```

9. Implementation Phases

Phase 1: Foundation (Week 1) -- "Store Everything"

Goal: Start capturing alarm events from day one.

Task	File	Lines	Priority
Create SQLite schema	`backend/db/init_alarm_db.py`	new	P1
Alarm type taxonomy constants	`backend/alarm_constants.py`	new	P1
Pydantic models	`backend/alarm_models.py`	new	P1
Ingestion poller thread	`backend/alarm_ingestion.py`	new	P1
Register poller in servidor_ac	`servidor_academia.py`	~line 1318	P1
Basic event query endpoint	`servidor_academia.py`	before 6495	P1
Active alarms endpoint	`servidor_academia.py`	before 6495	P1

Deliverable: Alarms being stored in SQLite. Query-able via API. No frontend yet.

Validation: After 24 hours of running, query /api/noc/alarms/events?period=24h and verify events are being captured with correct classification and timing.

Phase 2: Core Metrics (Week 2) -- "Measure It"

Goal: MTTR, MTBF, availability calculations working.

Task	File	Priority
Metrics computation functions	`backend/alarm_metrics.py`	P1
Metrics API endpoints (summary, mttr, mtb	`servidor_academia.py`	P1
Metrics cache table population (backgroun	`backend/alarm_ingestion.py`	P2
CFE outage tracking	`backend/alarm_ingestion.py`	P2
CFE outage endpoint	`servidor_academia.py`	P2

Deliverable: All core metric endpoints returning real data.

Phase 3: Dashboard UI (Week 3) -- "See It"

Goal: Frontend dashboard with KPIs, charts, alarm log.

Task	File	Priority
TypeScript types	`src/.../alarm-metrics/types.ts`	P1
Data fetching hooks	`src/.../alarm-metrics/hooks/`	P1
AlarmMetricsSection container	`src/.../AlarmMetricsSection.tsx`	P1
MetricsDashboard (KPI cards)	`src/.../alarm-metrics/MetricsDashboard.t	P1
AlarmLog (event table)	`src/.../alarm-metrics/AlarmLog.tsx`	P1
ActiveAlarms (with ack/escalate)	`src/.../alarm-metrics/ActiveAlarms.tsx`	P1
Register section in index.tsx	`src/pages/xcien2/index.tsx`	P1

Deliverable: Working dashboard accessible from portal sidebar.

Phase 4: Advanced Analytics (Week 4) -- "Understand It"

Goal: Trend charts, top offenders, correlation engine.

Task	File	Priority
TrendChart (SVG line chart)	`src/.../alarm-metrics/TrendChart.tsx`	P2
AvailabilityChart (bar chart)	`src/.../alarm-metrics/AvailabilityChart.	P2
TopOffenders list	`src/.../alarm-metrics/TopOffenders.tsx`	P2
Correlation engine	`backend/alarm_ingestion.py`	P2
Vendor/type breakdown charts	`src/.../alarm-metrics/MetricsDashboard.t	P2
Trend endpoints	`servidor_academia.py`	P2
Top offenders endpoint	`servidor_academia.py`	P2

Deliverable: Full analytics with trend analysis and cascade suppression.

Phase 5: Operations (Week 5-6) -- "Act On It"

Goal: Shift reports, SLA tracking, escalation, Telegram integration.

Task	File	Priority
Shift report generation	`backend/alarm_metrics.py`	P2
Shift report endpoint + PDF	`servidor_academia.py`	P2
ShiftReport component	`src/.../alarm-metrics/ShiftReport.tsx`	P2
SLA tracker endpoint	`servidor_academia.py`	P3
SlaTracker component	`src/.../alarm-metrics/SlaTracker.tsx`	P3
CfeOutageTracker component	`src/.../alarm-metrics/CfeOutageTracker.t	P2
Auto-escalation background worker	`backend/alarm_ingestion.py`	P3
Telegram escalation notifications	`backend/alarm_ingestion.py`	P3

Deliverable: Complete operational NOC alarm management system.

10. File Inventory

New Files

```
backend/
  alarm_constants.py          -- Alarm taxonomy, severity levels, thresholds
  alarm_models.py             -- Pydantic request/response models
  alarm_ingestion.py          -- Poller daemon, state detection, correlation engine
  alarm_metrics.py            -- MTTR/MTBF/availability computation functions
  db/
    init_alarm_db.py          -- SQLite schema creation script
    alarm_log.db              -- SQLite database (created at runtime)
```

```
src/pages/xcien2/sections/
  AlarmMetricsSection.tsx     -- Main section component
  alarm-metrics/
    types.ts                  -- TypeScript interfaces
    constants.ts              -- Colors, thresholds, severity maps
    utils.ts                  -- Formatters, duration calculators
    hooks/
      useAlarmData.ts         -- Data fetching + polling
      useMetrics.ts           -- Metrics API with caching
    AlarmLog.tsx              -- Event log table
    ActiveAlarms.tsx          -- Active alarm cards
    MetricsDashboard.tsx      -- KPI cards + charts
    AvailabilityChart.tsx     -- Availability by city/host
    TrendChart.tsx            -- Time-series trend charts
    TopOffenders.tsx          -- Worst-performing sites table
    CfeOutageTracker.tsx      -- CFE outage timeline
    ShiftReport.tsx           -- Shift handover display
    SlaTracker.tsx            -- SLA compliance dashboard
```

Modified Files

```
backend/servidor_academia.py
  - Import alarm modules (~line 50)
  - Start ingestion thread (~line 1318, after watchdog)
  - Add 15+ new API endpoints (before line 6495 catch-all)

src/pages/xcien2/index.tsx
  - Register AlarmMetricsSection in SECTIONS array
  - Add sidebar entry under Infraestructura group
```

11. Operational Considerations

11.1 Database Maintenance

```
# Weekly maintenance job (add to alarm_ingestion.py)
def weekly_maintenance():
    """Run every Sunday at 03:00 AM."""
    conn = sqlite3.connect(DB_PATH)

    # 1. Purge old full snapshots (keep 90 days)
    conn.execute("""
        DELETE FROM host_snapshots
        WHERE state_changed = 0
        AND snapshot_time < datetime('now', '-90 days')
    """)

    # 2. Purge old cleared alarms from active table
    conn.execute("""
        DELETE FROM alarm_active
        WHERE correlation_id IN (
            SELECT correlation_id FROM alarm_events
            WHERE event_type = 'clear'
            AND event_time < datetime('now', '-7 days')
        )
    """)

    # 3. Vacuum to reclaim space
    conn.execute("VACUUM")

    # 4. Refresh metrics cache
    recompute_metrics_cache(conn)

    conn.commit()
    conn.close()
```

11.2 Performance Targets

Metric	Target	Rationale
Poll cycle	< 5 sec for 76 hosts	30s interval, must complete well within
Event insert	< 10 ms	SQLite WAL mode, single writer
Metrics query (24h)	< 200 ms	Indexed queries on recent data
Metrics query (90d)	< 2 sec	Acceptable for dashboard load
Dashboard initial load	< 1 sec	2 parallel API calls
Active alarms refresh	< 500 ms	30s polling from frontend
DB size after 1 year	< 100 MB	With snapshot compression

11.3 SQLite WAL Mode

```
# Enable WAL mode for concurrent reads during writes
conn = sqlite3.connect(DB_PATH)
conn.execute("PRAGMA journal_mode=WAL")
conn.execute("PRAGMA synchronous=NORMAL")  -- Faster writes, safe with WAL
conn.execute("PRAGMA cache_size=-64000")   -- 64MB cache
```

This allows the ingestion thread to write while FastAPI request handlers read concurrently -- critical for a monolith architecture.

11.4 Graceful Degradation

```
If NOCBoard Energia API (9404) is down:
  1. Poller logs warning, skips poll cycle
  2. Frontend shows "Data Source Unavailable" banner
  3. Historical data still queryable from SQLite
  4. Active alarms table retains last known state

If SQLite is corrupted:
  1. alarm_ingestion.py detects on startup
  2. Renames corrupt file to alarm_log.db.corrupt.{timestamp}
  3. Re-creates fresh database from schema
  4. Logs critical error + Telegram notification
  5. Historical data lost but system continues
```

12. Telecom Industry Best Practices Applied

12.1 ITU-T X.733 Compliance

The alarm model directly maps to X.733 managed object attributes:

* alarm_type -- maps to X.733 alarm type (communicationsAlarm, environmentalAlarm, etc.)
* perceived_severity -- follows X.733 severity scale (critical, major, minor, warning, indeterminate, cleared)
* probable_cause -- uses X.733 probable cause values (powerProblem, equipmentMalfunction, etc.)
* specific_problem -- free-text description following X.733 guidelines

12.2 TMN Fault Management

The system implements TMN Fault Management (FM) layer concepts:

* Event Detection: SNMP polling via NOCBoard (already implemented)
* Event Correlation: Cascade detection, CFE grouping, flap damping
* Fault Diagnosis: Alarm classification by type and probable cause
* Fault Resolution Tracking: State machine from raise through acknowledge to clear
* Reporting: MTTR, MTBF, availability, trend analysis

12.3 SNMP Trap Readiness

While the current system uses polling (NOCBoard polls SNMP, we poll NOCBoard), the data model supports future trap-based ingestion:

* event_time vs ingested_at distinguishes when the event occurred vs when we learned about it
* raw_data stores the full source data regardless of delivery mechanism
* Trap receiver could write directly to alarm_events table using the same schema

12.4 NOC Operational Procedures

The system encodes telecom NOC best practices:

* Alarm Windowing: 2-minute cascade window prevents alert storms

* Flap Damping: Prevents operator fatigue from unstable links
* Escalation Matrix: Time-based automatic escalation with three levels
* Shift Handover: Structured reports ensure continuity between shifts
* SLA Tracking: Availability targets aligned with Mexican telecom standards
* Post-Mortem Integration: Links to existing incident system (db/incidentes.json)
* Root Cause Analysis: Top offenders and trend analysis enable preventive maintenance

13. Integration Points

13.1 Existing System Connections

```
NOCBoard Energia (localhost:9404)
  <- Poller reads /api/hosts, /api/host/:id, /api/alerts, /api/events
  <- Every 30 seconds
  <- Fallback: ~/Library/Application Support/NOCBoardEnergia/hosts.json

Telegram (@xcien_nocboard_bot)
  -> Escalation notifications (Level 1)
  -> Shift report PDFs
  -> Cascade summary alerts (instead of N individual alerts)

Telegram (@jmmc2026_bot)
  -> Escalation notifications (Level 2, 3)
  -> Shift report PDFs

Incident System (db/incidentes.json)
  -> Link alarms to incidents via ticket_id field
  -> Auto-create incident for sustained P1 alarms

XCIEN 2.0 Portal (React frontend)
  -> New section in sidebar: "Alarmas y Metricas"
  -> Shares theme system, dark mode, inline style patterns
  -> Fetches from /api/noc/alarms/* endpoints

Existing NOC Section (NocSection.tsx)
  -> Can embed ActiveAlarms summary widget
  -> Can show alarm count in board status cards
  -> Cross-link: clicking an alarm in NocSection opens AlarmMetricsSection
```

13.2 Future Integration Path

```
Phase 6 (future):
  - Odoo project.task auto-creation for P1/P2 alarms
  - Observium integration (network device alarms alongside power alarms)
  - NOCBoard Datos + WL alarm ingestion (same engine, different API ports)
  - Grafana/external BI export via CSV endpoint
  - Mobile push notifications via PWA service worker
  - AI-powered anomaly detection (Claude analysis of alarm patterns)
```

14. Risk Assessment

Risk	Probability	Impact	Mitigation
NOCBoard API instability	Medium	Medium	File fallback exists; poller s
SQLite write contention	Low	Low	WAL mode; single writer thread
Large cascade event (69+ hosts	Medium	High	Correlation engine suppresses;
Storage growth beyond expectat	Low	Low	Snapshot compression + 90-day
Poll cycle exceeds interval	Low	Medium	5s target for 76 hosts; timeou
Frontend performance with larg	Low	Medium	Pagination (500 default), serv
Schema migration needed	Medium	Low	SQLite schema versioning via `

Appendix A: Quick Reference -- Endpoint Summary

```
GET  /api/noc/alarms/events                    -- Historical alarm log
GET  /api/noc/alarms/events/{id}               -- Single event detail
GET  /api/noc/alarms/active                    -- Current active alarms
POST /api/noc/alarms/active/{id}/acknowledge   -- Acknowledge alarm
POST /api/noc/alarms/active/{id}/escalate      -- Manual escalation
POST /api/noc/alarms/active/{id}/notes         -- Add operator notes
GET  /api/noc/alarms/metrics/summary           -- KPI summary
GET  /api/noc/alarms/metrics/availability      -- Availability by group
GET  /api/noc/alarms/metrics/mttr              -- MTTR breakdown
GET  /api/noc/alarms/metrics/mtbf              -- MTBF breakdown
GET  /api/noc/alarms/metrics/trends            -- Time-series trends
GET  /api/noc/alarms/top-offenders             -- Worst sites ranked
GET  /api/noc/alarms/cfe-outages               -- CFE power events
GET  /api/noc/alarms/sla                       -- SLA compliance
GET  /api/noc/alarms/shift-report              -- Shift handover data
GET  /api/noc/alarms/shift-report/pdf          -- Shift report as PDF
```

Appendix B: Severity Color Reference

```
Critical (#ff3366) -- Red pulse     -- Immediate action
Major    (#ff8800) -- Orange        -- Action within 15 min
Minor    (#ffcc00) -- Yellow        -- Action within 4 hours
Warning  (#88aaff) -- Light blue    -- Monitor
Cleared  (#00ff88) -- Green         -- Resolved
```